BUS RESERVATION SYSTEM

A PROJECT REPORT

Submitted by

DHRUV DIXIT [Reg No.: RA2511003030038]

PRERAK SHARMA [Reg No.: RA2511003030004]

Under the guidance of

Dr. UMA MEENA

(Associate Professor, Department of Computer Science & Engineering)

in partial fulfilment for the award of the degree

BACHELOR OF TECHNOLOGY

in

COWUTER SCIENCE & ENGINEERING

Of

FACULTY OF ENGINEERING AND TECHNOLOGY

DELHI-NCR CAMPUS, GHAZLABAD (UP)

SRM NSTITUTE OF SCIENCE AND TECHNOLOGY, NCR CAMPUS

TABLE OF CONTENT

S.No	Topic
1	Abstract
2	Introduction
3	Literature Survey
4	Methodology Used
5	Hardware/Software Requirements
6	Implementation
7	Result
8	Advantages
9	Limitations
10	Conclusions
11	References

ABSTRACT

The Bus Reservation System is a console-based application developed using the C++ programming language to demonstrate the practical implementation of object-oriented programming concepts. The main objective of this project is to automate the basic process of bus seat reservation and replace manual record keeping with a simple computerized system.

The system is designed using classes such as Bus, User, Booking, and ReservationSystem, where each class is responsible for managing a specific part of the application. The program allows the user to add bus details, register users, view available buses, check seat availability, book seats, cancel reservations, and display booking records. All data related to buses, users, and bookings is stored in structured form using dynamic containers from the C++ Standard Library. Before confirming a reservation, the system checks whether the bus exists, the user exists, and the selected seat is available, which prevents invalid bookings and ensures accurate seat management. The program is implemented as a menu-driven console application using loops and conditional statements to control the flow of execution.

The project is organized into multiple header and source files and compiled using the g++ compiler with CMake as the build system, making the program modular and easy to maintain. This project helps in understanding the practical use of object-oriented programming, data handling, and program organization while solving a real-world type problem related to reservation management. The system can be further improved by adding file storage, database connectivity, graphical interface, or online booking features in the future.

INTRODUCTION

The Bus Reservation System is a simple software application developed using the C++ programming language to demonstrate the use of object-oriented programming for managing reservation operations. The main objective of this project is to replace manual record keeping with a computerized system that can store and manage information about buses, users, and seat bookings in an organized manner.

In traditional reservation methods, ticket booking and seat management are often handled manually, which can lead to errors, confusion, and difficulty in tracking seat availability. A computerized system helps to reduce these problems by maintaining data in a structured format and performing automatic checks before completing any operation.

The Bus Reservation System allows the user to add bus details, register users, view available buses, check seat availability, reserve seats, cancel bookings, and display booking records. Before a seat is booked, the system verifies that the selected bus exists, the user exists, and the seat is available. This prevents invalid bookings and ensures that seat information remains accurate. The project is implemented using object-oriented programming concepts in C++. Different parts of the system are represented using classes such as Bus, User, Booking, and ReservationSystem. Each class is responsible for handling specific data and operations, which makes the program easier to understand and maintain. The program uses dynamic containers from the C++ Standard Library to store data, and loops and conditional statements are used to control the program flow. A menu-driven interface is provided so that the user can perform different operations repeatedly until the program is exited.

The project is organized into separate header and source files and is compiled using the g++ compiler with CMake as the build system. This structure makes the program modular and easier to build and modify. Overall, the Bus Reservation System is a simple educational project that demonstrates how C++ programming can be used to design a small management system. It helps in understanding the practical use of object-oriented programming, data handling, and program organization while solving a real-world type problem related to reservation management.

LITERATURE SURVEY

A bus reservation system is a common software application used to manage passenger transportation and automate the process of booking bus seats. In traditional methods, passengers had to visit booking counters and manually reserve tickets, which was time-consuming and often led to errors in record keeping.

With the use of computer programming, reservation systems can be developed to simplify the booking process and maintain data in a more organized way. Many academic projects and software applications have been developed to demonstrate computerized reservation systems using programming languages such as C++.

A basic bus reservation system usually provides functions like adding bus details, viewing available buses, checking seat availability, booking seats, and cancelling reservations.

These systems store information about buses, users, and seat allocations, allowing the program to manage booking data efficiently.

Object-oriented programming languages such as C++ are commonly used to implement reservation systems because they support modular programming, data encapsulation, and clear separation of functionality.

In a typical design, the system is divided into multiple classes such as Bus, User, Booking, and ReservationSystem, where each class handles a specific part of the application.

This approach makes the program easier to understand, modify, and extend.

Earlier versions of reservation systems were usually console-based applications designed for learning and small-scale management.

These systems use menu-driven interfaces, loops, and conditional statements to control the program flow and allow the user to perform different operations repeatedly.

Data is stored in memory using arrays or dynamic containers, and the program checks seat availability before confirming a booking to avoid errors.

Modern reservation systems may include graphical interfaces, database storage, and network-based access, but simple console-based implementations are still useful for learning programming concepts and understanding real-world problem solving.

From the literature, it is clear that developing a bus reservation system using C++ is a suitable way to understand object-oriented programming, data handling, and program design while implementing a practical application related to transportation management.

METHODOLOGY USED

The development of the Bus Reservation System follows an object-oriented programming methodology using C++.

The system is designed by dividing the program into multiple classes, where each class handles a specific part of the reservation process.

The main classes used in the program are Bus, User, Booking, and ReservationSystem.

The methodology followed in the implementation includes the following steps:

- Creating bus objects with route details and seat capacity
- Creating user records with unique user IDs
- Storing buses, users, and bookings using dynamic containers (vector)
- Displaying the list of available buses to the user
- Showing seat layout for the selected bus
- Checking seat availability before confirming a booking
- Creating a booking record when a seat is reserved
- Allowing cancellation of a reserved seat
- Displaying all booking details when requested

The ReservationSystem class controls the overall flow of the program using a menu-driven interface.

Loops are used to keep the program running until the user chooses to exit, and conditional statements are used to validate user input and prevent invalid operations such as booking an already reserved seat or selecting a non-existent bus.

The project is organized into multiple header and source files, and the program is compiled using CMake to manage the build process efficiently.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware: **(**recommended**)**

- Intel i3 or higher processor
- 4 GB RAM
- Keyboard
- 500 MB free disk space
- Monitor

Software:

- Operating System: Windows / Linux / macOS
- Programming Language: C++, bash
- Compiler: g++ (recommended: cmake)
- IDE: Code::Blocks / Visual Studio Code / Dev C++/ Neovim
- Libraries used: <iostream>, <limits>, <string>, <vector>

IMPLEMENTATION AND PROGRAMMING

The Bus Reservation System is implemented using object-oriented programming in C++.

The program is structured using multiple classes such as Bus, User, Booking, and ReservationSystem to separate different responsibilities of the system.

The project is built using the C++17 standard and compiled using g++ with CMake as the build system.

The system provides the following main functionalities:

- Add new buses with route details and seat capacity
- Add users with unique user ID and name
- Display available buses and their details
- Show seat layout for a selected bus
- Book a seat for a user after checking availability
- Cancel a booked seat
- Store and display all booking records

Each bus maintains its own seat list using a vector, where a seat stores the user ID of the passenger who booked it.

Before booking, the program verifies that the bus exists, the user exists, and the selected seat is valid and available.

The Reservation System class manages all buses, users, and bookings using dynamic containers from the C++ Standard Library.

Loops and conditional statements are used to control the menu-driven interface, allowing the program to run continuously until the user chooses to exit.

The program is organized into separate header and source files for each class, and the project is compiled using CMake to ensure proper build management.

PROJECT STRUCTURE

```
BusReservationSystem/
├── include/              # Header files
│   ├── Booking.h
│   ├── Bus.h
│   ├── ReservationSystem.h
│   └── User.h
│
├── src/                  # Source files
│   ├── booking/
│   │   └── Booking.cpp
│   ├── bus/
│   │   └── Bus.cpp
│   ├── system/
│   │   └── ReservationSystem.cpp
│   ├── user/
│   │   └── User.cpp
│   └── main.cpp
│
├── CMakeLists.txt
└── README.md
```

src/main.cpp

```cpp
#include <iostream>
#include <limits>
#include "ReservationSystem.h"

using namespace std;

int main() {

    ReservationSystem system;
    int choice;

    while(true) {

        cout << "\n--- Bus Reservation System ---\n";
        cout << "1 Add Bus\n";
        cout << "2 View Buses\n";
        cout << "3 Add User\n";
        cout << "4 Book Seat\n";
        cout << "5 Cancel Booking\n";
        cout << "6 View Bookings\n";
        cout << "7 Exit\n";
        cout << "Enter choice: ";

        if(!(cin >> choice)) {
            cout << "Invalid input. Please enter a number.\n";

            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }

        switch(choice) {

            case 1:
                system.addBus();
                break;

            case 2:
                system.viewBuses();
                break;

            case 3:
                system.addUser();
                break;

            case 4:
                system.bookSeat();
                break;

            case 5:
```

```
                system.cancelBooking();
                break;

            case 6:
                system.viewBookings();
                break;

            case 7:
                cout << "Exiting program...\n";
                return 0;

            default:
                cout << "Invalid choice. Try again.\n";
        }
    }
}

```

include/ReservationSystem.h

```cpp
#ifndef RESERVATIONSYSTEM_H
#define RESERVATIONSYSTEM_H

#include <vector>
#include "Bus.h"
#include "User.h"
#include "Booking.h"

class ReservationSystem {
private:
    std::vector<Bus> buses;
    std::vector<User> users;
    std::vector<Booking> bookings;

public:
    ReservationSystem() = default;

    void addBus();
    void viewBuses();

    void addUser();

    void bookSeat();
    void cancelBooking();
    void viewBookings();
};

#endif

```

src/system/ReservationSystem.cpp

```cpp
#include "ReservationSystem.h"
#include <iostream>
#include <limits>

using namespace std;

void ReservationSystem::addBus() {
    int id, seats;
    string src, dest;

    cout << "Bus ID: ";
    if(!(cin >> id)) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Invalid ID\n";
        return;
    }

    for(auto &b : buses) {
        if(b.getBusID() == id) {
            cout << "Bus ID already exists\n";
            return;
        }
    }

    cout << "Source: ";
    cin >> src;

    cout << "Destination: ";
    cin >> dest;

    cout << "Total Seats: ";
    if(!(cin >> seats) || seats <= 0) {
        cout << "Invalid seat count\n";
        return;
    }

    buses.push_back(Bus(id, src, dest, seats));
    cout << "Bus added successfully\n";
}

void ReservationSystem::viewBuses() {

    if(buses.empty()) {
        cout << "No buses available\n";
        return;
    }

    for(auto &bus : buses) {
        bus.displayBus();
```

```
        cout << endl;
    }
}

void ReservationSystem::addUser() {
    int id;
    string name;

    cout << "User ID: ";
    if(!(cin >> id)) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Invalid ID\n";
        return;
    }

    for(auto &u : users) {
        if(u.getUserID() == id) {
            cout << "User ID already exists\n";
            return;
        }
    }

    cout << "Name: ";
    cin >> name;

    users.push_back(User(id, name));

    cout << "User added successfully\n";
}

void ReservationSystem::bookSeat() {

    if(buses.empty()) {
        cout << "No buses available\n";
        return;
    }

    if(users.empty()) {
        cout << "No users registered\n";
        return;
    }

    int busID, seat, userID;

    cout << "Enter Bus ID: ";
    cin >> busID;

    Bus* foundBus = nullptr;

    for(auto &bus : buses) {
        if(bus.getBusID() == busID) {
            foundBus = &bus;
            break;
```

```
        }
    }

    if(foundBus == nullptr) {
        cout << "Bus not found\n";
        return;
    }

    foundBus->showSeats();

    cout << "Enter Seat Number: ";
    cin >> seat;

    cout << "Enter User ID: ";
    cin >> userID;

    bool userExists = false;

    for(auto &u : users) {
        if(u.getUserID() == userID) {
            userExists = true;
            break;
        }
    }

    if(!userExists) {
        cout << "User not found\n";
        return;
    }

    if(foundBus->bookSeat(seat, userID)) {

        int bookingID = bookings.size() + 1;

        bookings.push_back(
            Booking(bookingID, userID, busID, seat)
        );

        cout << "Booking Successful\n";
    }
    else {
        cout << "Seat unavailable or invalid\n";
    }
}

void ReservationSystem::cancelBooking() {

    if(buses.empty()) {
        cout << "No buses available\n";
        return;
    }

    int busID, seat;

```

```
    cout << "Bus ID: ";
    cin >> busID;

    cout << "Seat Number: ";
    cin >> seat;

    for(auto &bus : buses) {

        if(bus.getBusID() == busID) {

            bus.cancelSeat(seat);

            for(auto it = bookings.begin(); it != bookings.end(); ++it) {
                if(it->getBusID() == busID && it->getSeatNumber() == seat) {
                    bookings.erase(it);
                    break;
                }
            }

            cout << "Booking Cancelled\n";
            return;
        }
    }

    cout << "Bus not found\n";
}

void ReservationSystem::viewBookings() {

    if(bookings.empty()) {
        cout << "No bookings found\n";
        return;
    }

    for(auto &b : bookings) {
        b.displayBooking();
        cout << endl;
    }
}

```

include/User.h

```cpp
#ifndef USER_H
#define USER_H

#include <string>

class User {
private:
    int userID;
    std::string name;

public:
    User(int id, const std::string& n);

    int getUserID() const;
    std::string getName() const;

    void displayUser() const;
};

#endif

```

src/user/User.cpp

```cpp
#include "User.h"
#include <iostream>

User::User(int id, const std::string& n) {
    userID = id;
    name = n;
}

int User::getUserID() const {
    return userID;
}

std::string User::getName() const {
    return name;
}

void User::displayUser() const {
    std::cout << "User ID: " << userID << std::endl;
    std::cout << "Name: " << name << std::endl;
}

```

include/Bus.h

```cpp
#ifndef BUS_H
#define BUS_H

#include <string>
#include <vector>

class Bus {
private:
    int busID;
    std::string source;
    std::string destination;
    int totalSeats;
    std::vector<int> seats;

public:
    Bus(int id, std::string src, std::string dest, int seatsCount);

    int getBusID() const;
    int getTotalSeats() const;

    void displayBus() const;

    void showSeats() const;
    bool bookSeat(int seatNumber, int userID);
    bool cancelSeat(int seatNumber);
};

#endif

```

src/bus/Bus.cpp

```cpp
#include "Bus.h"
#include <iostream>

Bus::Bus(int id, std::string src, std::string dest, int seatsCount) {
    busID = id;
    source = src;
    destination = dest;
    totalSeats = seatsCount;

    seats.resize(totalSeats, 0);
}

int Bus::getBusID() const {
    return busID;
}

int Bus::getTotalSeats() const {
    return totalSeats;
}

void Bus::displayBus() const {
    std::cout << "Bus ID: " << busID << std::endl;
    std::cout << "Route: " << source << " -> " << destination << std::endl;
    std::cout << "Total Seats: " << totalSeats << std::endl;
}

void Bus::showSeats() const {

    std::cout << "\nSeat Layout (X = booked)\n\n";

    for(int i = 0; i < totalSeats; i++) {

        if(seats[i] == 0)
            std::cout << "[" << i+1 << "]\t";
        else
            std::cout << "[X]\t";

        if((i + 1) % 4 == 0)
            std::cout << std::endl;
    }

    std::cout << std::endl;
}
bool Bus::bookSeat(int seatNumber, int userID) {

    if(seatNumber < 1 || seatNumber > totalSeats) {
        std::cout << "Invalid seat number\n";
        return false;
    }

```

```
    if(userID <= 0) {
        std::cout << "Invalid user ID\n";
        return false;
    }

    if(seats[seatNumber - 1] != 0) {
        std::cout << "Seat already booked\n";
        return false;
    }

    seats[seatNumber - 1] = userID;
    return true;
}

bool Bus::cancelSeat(int seatNumber) {

    if(seatNumber < 1 || seatNumber > totalSeats) {
        std::cout << "Invalid seat number\n";
        return false;
    }

    if(seats[seatNumber - 1] == 0) {
        std::cout << "Seat already empty\n";
        return false;
    }

    seats[seatNumber - 1] = 0;
    return true;
}

```

include/Booking.h

```cpp
#ifndef BOOKING_H
#define BOOKING_H

class Booking {
private:
    int bookingID;
    int userID;
    int busID;
    int seatNumber;

public:
    Booking(int bID, int uID, int bus, int seat);

    int getBookingID() const;
    int getUserID() const;
    int getBusID() const;
    int getSeatNumber() const;

    void displayBooking() const;
};

#endif

```

src/booking/Booking.cpp

```cpp
#include "Booking.h"
#include <iostream>

Booking::Booking(int bID, int uID, int bus, int seat) {
    bookingID = bID;
    userID = uID;
    busID = bus;
    seatNumber = seat;
}

int Booking::getBookingID() const {
    return bookingID;
}

int Booking::getUserID() const {
    return userID;
}

int Booking::getBusID() const {
    return busID;
}

int Booking::getSeatNumber() const {
    return seatNumber;
}

void Booking::displayBooking() const {
    std::cout << "Booking ID: " << bookingID << std::endl;
    std::cout << "User ID: " << userID << std::endl;
    std::cout << "Bus ID: " << busID << std::endl;
    std::cout << "Seat: " << seatNumber << std::endl;
}

```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)
project(BusReservationSystem)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

include_directories(include)

file(GLOB_RECURSE SRC "src/*.cpp")

add_executable(bus ${SRC})

```

Compile and Run

```
# Using CMake (recommended)
cd BusReservationSystem
cmake -S . -B build
cmake --build build
./build/bus

# or (without CMake)
cd BusReservationSystem

g++ -std=c++17 -Iinclude \
src/main.cpp \
src/booking/Booking.cpp \
src/bus/Bus.cpp \
src/system/ReservationSystem.cpp \
src/user/User.cpp \
-o bus

./bus
```

OUTPUT

```
┌─ pwe in 📂~/Code/class/BusReservationSystem ⚡ main
└─→  ./build/bus

--- Bus Reservation System ---
1 Add Bus
2 View Buses
3 Add User
4 Book Seat
5 Cancel Booking
6 View Bookings
7 Exit
Enter choice: 1
Bus ID: 101
Source: pune
Destination: delhi
Total Seats: 40
Bus added successfully

--- Bus Reservation System ---
1 Add Bus
2 View Buses
3 Add User
4 Book Seat
5 Cancel Booking
6 View Bookings
7 Exit
Enter choice: 2
Bus ID: 101
Route: pune -> delhi
Total Seats: 40


--- Bus Reservation System ---
1 Add Bus
2 View Buses
3 Add User
4 Book Seat
5 Cancel Booking
6 View Bookings
7 Exit
Enter choice: 3
User ID: 001
Name: Prerak
User added successfully
```

```
--- Bus Reservation System ---
1 Add Bus
2 View Buses
3 Add User
4 Book Seat
5 Cancel Booking
6 View Bookings
7 Exit
Enter choice: 4
Enter Bus ID: 101

Seat Layout (X = booked)

[1]     [2]     [3]     [4]
[5]     [6]     [7]     [8]
[9]     [X]     [11]    [12]
[13]    [14]    [15]    [16]
[17]    [18]    [19]    [20]
[21]    [22]    [23]    [24]
[25]    [26]    [27]    [28]
[29]    [30]    [31]    [32]
[33]    [34]    [35]    [36]
[37]    [38]    [39]    [40]

Enter Seat Number: 4
Enter User ID: 001
Booking Successful
```

```
--- Bus Reservation System ---
1 Add Bus
2 View Buses
3 Add User
4 Book Seat
5 Cancel Booking
6 View Bookings
7 Exit
Enter choice: 6
Booking ID: 1
User ID: 1
Bus ID: 101
Seat: 10

Booking ID: 2
User ID: 1
Bus ID: 101
Seat: 4


--- Bus Reservation System ---
1 Add Bus
2 View Buses
3 Add User
4 Book Seat
5 Cancel Booking
6 View Bookings
7 Exit
Enter choice: 7
Exiting program...
┌─ pwe in ~/Code/class/BusReservationSystem ⚡ main
└─→
```

RESULT

The Bus Reservation System mini project was successfully designed and implemented using the C++ programming language with an object-oriented approach.

The objective of the project was to develop a console-based application that can manage basic bus reservation operations such as adding buses, registering users, booking seats, cancelling bookings, and displaying reservation details.

The program was developed using multiple classes including Bus, User, Booking, and ReservationSystem, where each class handles a specific part of the system.

The system stores buses, users, and booking records using dynamic containers from the C++ Standard Library, and ensures that all operations are validated before execution.

The application provides a menu-driven interface that allows the user to perform different actions repeatedly until the program is exited.

Before reserving a seat, the system checks whether the bus exists, the user exists, and the selected seat is available.

Seat information is maintained for each bus, and once a seat is booked it cannot be reserved again until it is cancelled.

The project demonstrates the practical use of object-oriented programming concepts such as classes, objects, encapsulation, and modular design.

It also shows the use of loops, conditional statements, and input validation to control the program flow and prevent incorrect operations.

The project was organized into multiple source and header files and compiled using CMake with the g++ compiler, which makes the program easier to build and maintain.

The system works correctly for basic reservation management and can be further improved by adding features such as file storage, database support, GUI or online booking functionality.

ADVANTAGES

- Simple and easy to use menu-driven interface
- Faster booking compared to manual record keeping
- Prevents double booking of seats
- Better organization of bus, user, and booking data
- Easy to modify and extend due to object-oriented design
- Works on multiple operating systems (Windows / Linux / macOS)
- Requires very less memory and system resources

The Bus Reservation System makes the process of managing bus bookings easier compared to manual methods.

The program allows the user to add buses, register users, book seats, cancel reservations, and view booking details through a simple console interface.

The system automatically checks seat availability before booking, which prevents errors such as reserving the same seat multiple times.

All data related to buses, users, and bookings is stored in structured form using classes, which makes the program easy to understand and maintain.

Since the project is developed using object-oriented programming in C++, the code is divided into separate classes and files.

This makes the program flexible and allows new features to be added in the future without changing the entire system.

The program can be compiled using g++ and built using CMake, so it can run on different operating systems without major changes.

Compared to manual ticket booking, this system reduces time, avoids confusion, and provides a more organized way to manage reservations.

LIMITATIONS

- Works only in console mode (no graphical interface)
- Data is not saved permanently after program exits
- No database or file storage support
- No online booking or network connectivity
- Limited error handling for complex inputs
- Suitable only for small-scale reservation management

The Bus Reservation System is a simple console-based application, so it does not provide a graphical user interface.

All operations are performed using text input and output, which may not be as user-friendly as modern software applications.

The program stores all data in memory using vectors, so the information about buses, users, and bookings is lost when the program is closed.

Since no file handling or database is used, the system cannot keep permanent records.

The system works only for basic reservation operations and does not support online booking, payment systems, or network-based access.

It is designed mainly for learning purposes and small-scale use.

The project also has limited validation and does not handle very complex input errors.

For large systems, more advanced features such as database connectivity, graphical interface, and multi-user support would be required.

Overall, the program is suitable for demonstrating object-oriented programming concepts, but it is not intended for real-world commercial use without further improvements.

CONCLUSION

The Bus Reservation System was successfully designed and implemented using the C++ programming language with an object-oriented approach.

The main objective of the project was to create a simple console-based application that can manage basic bus reservation operations in an organized and efficient way.

The system allows the user to add bus details, register users, view available buses, check seat availability, book seats, cancel bookings, and display reservation records.

All operations are controlled through a menu-driven interface, making the program easy to use and understand.

During the development of this project, object-oriented programming concepts such as classes, objects, encapsulation, and modular design were applied.

The program was divided into different classes like Bus, User, Booking, and ReservationSystem, which makes the code structured and easy to maintain.

The project also demonstrates the use of vectors, loops, conditional statements, and input validation to control the program flow and prevent incorrect operations such as booking an already reserved seat or selecting an invalid bus.

The system was built using g++ and CMake, and the code was organized into multiple header and source files, which makes the project easier to compile and extend in the future.

Although the program is a simple console-based system, it clearly shows how programming can be used to automate manual reservation work and manage data more efficiently.

The project can be further improved by adding file storage, database support, graphical user interface, or online booking features.

Overall, the Bus Reservation System successfully demonstrates the practical implementation of C++ object-oriented programming concepts for solving a real-world type problem in a simple and structured way.

REFERENCES

(1) K. Ammar, M. Jalmoud, A. Boushehri and K. Fakhro, "A Real-time School Bus Tracking and Monitoring System," 2019 IEEE 10th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON), Vancouver, BC, Canada, 2019, pp. 0654- 0660, doi: 10.1109/IEMCON.2019.8936199.

(2) S. Akter, T. Islam, R. F. Olanrewaju and A. A. Binyamin, "A CloudBased Bus Tracking System Based on Internet-of-Things Technology," 2019 7th International Conference on Mechatronics Engineering (ICOM), Putrajaya, Malaysia, 2019, pp. 1-5, doi: 10.1109/ICOM47790.2019.8952037.

(3) T. Kavitha and G. Senbagavalli, "The Future of Travel in public Bus Service: How a Mobile Bus Ticketing System is Revolutionizing the Public Travel," 2023 International Conference on Applied Intelligence and Sustainable Computing (ICAISC), Dharwad, India, 2023, pp. 1-7, doi:10.1109/ICAISC58445.2023.10200016.

(4) D. Sarswat, "Real-Time AI Driven Shooting Posture Assessment and Correction for Professional and Military Training Using Machine Learning, OpenCV, and MediaPipe," 2024 4th International Conference on Technological Advancements in Computational Sciences (ICTACS), Tashkent, Uzbekistan, 2024, pp. 1192-1198, doi: 10.1109/ICTACS62700.2024.10841101.

RESEARCH PAPERS

1)https://ijarsct.co.in/Paper24493.pdf

2)https://www.researchgate.net/publication/379120241_Online_bus_reservation_system_project_report